Algoritmos

Um algoritmo é um conjunto de instruções que realizam uma tarefa.

Pesquisa binária

Se trata de uma forma otimizada de busca de dados em uma lista ordenada, usando a estratégia "dividir para conquistar". Resumidamente, com base no feedback de cada iteração do algoritmo de busca, calibramos o intervalo de dados que pode conter o objeto de interesse.

Vamos supor que você esteja procurando o nome de uma pessoa em uma agenda telefônica (que frase antiquada!). O nome começa com *K*. Você pode começar na primeira página da agenda e ir folheando até chegar aos *K*s. Porém você provavelmente vai começar pela metade, pois sabe que os *K*s estarão mais perto dali.

Ou suponha que esteja procurando uma palavra que começa com *O* em um dicionário. Novamente, você começa a busca pelo meio.

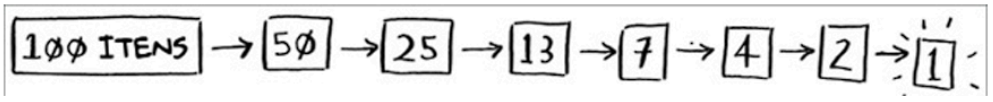
Agora, imagine que você entre no Facebook. Quando faz isso, o Facebook precisa verificar que você tem uma conta no site. Logo, ele procura seu nome de usuário em um banco de dados. Digamos que seu usuário seja karlmageddon. O Facebook poderia começar pelos *A*s e procurar seu nome – mas faz mais sentido que ele comece a busca pelo meio.

Vamos dizer que você esteja tentando encontrar um número de 1 a 100 e, a cada erro, recebe uma dica se a resposta é um valor "muito alto" ou "muito baixo" comparativamente ao número desejado.

Muito baixo, mas você eliminou *metade* dos números! Agora, você sabe que os números de 1 a 50 são muito baixos. Próximo chute: 75.

Muito alto, mas novamente você pode cortar metade dos números restantes! *Com a pesquisa binária, você chuta um número intermediário e elimina a metade dos números restantes a cada vez.* O próximo número é o 63 (entre 50 e 75).

Isso é a pesquisa binária. Você acaba de aprender um algoritmo! Aqui está a quantidade de números que você pode eliminar a cada tentativa.



Para encontrarmos o pior caso de uma busca binária, utilizamos a fórmula $\log_2(n)$. Em uma lista de 128 nomes, demoraríamos, no pior caso, 7 passos para encontrar o item desejado. Já o dobro de nomes (256) levariam, apenas, 8 passos, evidenciando a efetividade do método.
Caso utilizássemos busca linear, no pior caso, teríamos, respectivamente, 128 passos e 256 passos até encontrarmos o nome que estamos procurando.

Exemplo de algoritmo de busca binária em C++:

```
#include <iostream>
using namespace std;
```

```
int buscaBinaria(int vetor[5], int item, int ultimaPosicao){

    int baixo = 0;
    int alto = ultimaPosicao;

    while(baixo<=alto){
        int meio = (baixo+alto)/2;
        int chute = vetor[meio];

        if (chute == item){
            return meio; // Acertou!
        }

        if (chute > item){
            alto = meio-1; // Está na parte direita
        }

        if(chute < item){
            baixo = meio+1; // Está na parte esquerda
        }

        cout << "ITERAÇÃO! Alto = " << alto << ", Meio = " << meio <<
        ", Baixo = " << baixo << ", ITEM = " << item << endl;
    }

    return -1;
}

int main() {

    int vetor[5] = {1, 3, 5, 7, 9};
    int valor = buscaBinaria(vetor, 9, 4);

    if(valor == -1){
        cout << "Erro na busca!" << endl;
    } else {
        cout << vetor[valor] << endl;
    }

    return 0;
}
```

Tempo de execução

Quando o número máximo de tentativas é igual ao tamanho da lista, temos um exemplo de algoritmo com tempo **linear**. Já quando utilizamos o número máximo de

tentativas é igual a $\log_2(n)$, com n sendo o número máximo de tentativas, temos um exemplo de algoritmo com tempo **logarítmico**.

Exemplo utilizando algoritmos de busca:

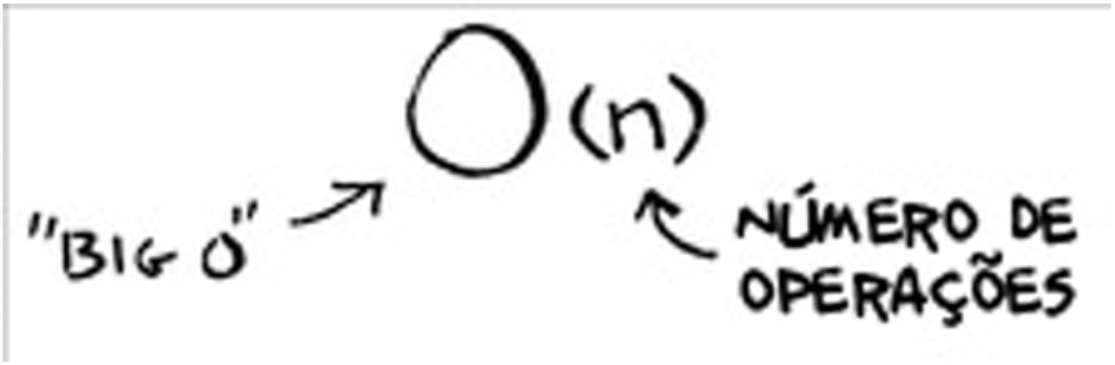
	PESQUISA SIMPLES	PESQUISA BINÁRIA
100 ELEMENTOS	100ms	7ms
10000 ELEMENTOS	10 segundos	14 ms
1,000,000,000 ELEMENTOS	11 dias	32 ms

Tempos de execução crescem com velocidades diferentes!

Notação Big O

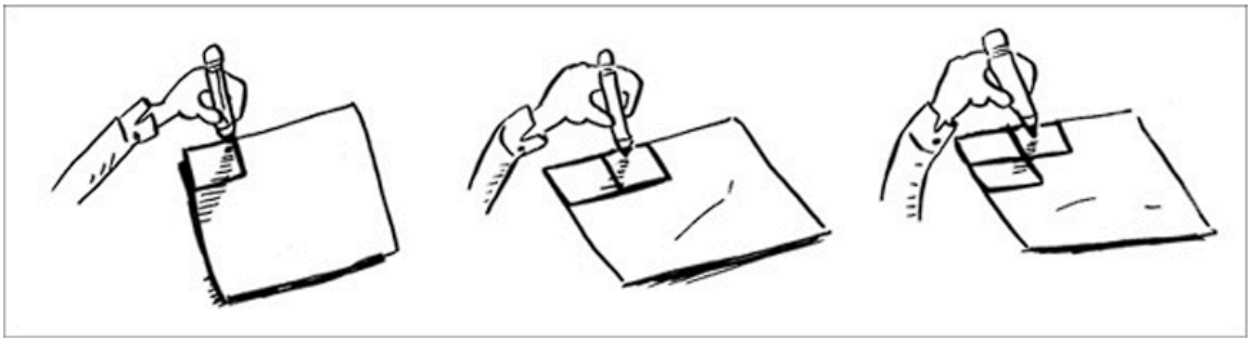
Essa é uma notação utilizada especificamente para dizer o quão rápido é um algoritmo. A notação Big O permite que você compare o número de operações e, portanto, não

utiliza medidas de tempo (segundos, milissegundos, etc) como métrica.
É chamada Big O justamente porque coloca-se um "grande O" na frente do número de operações.



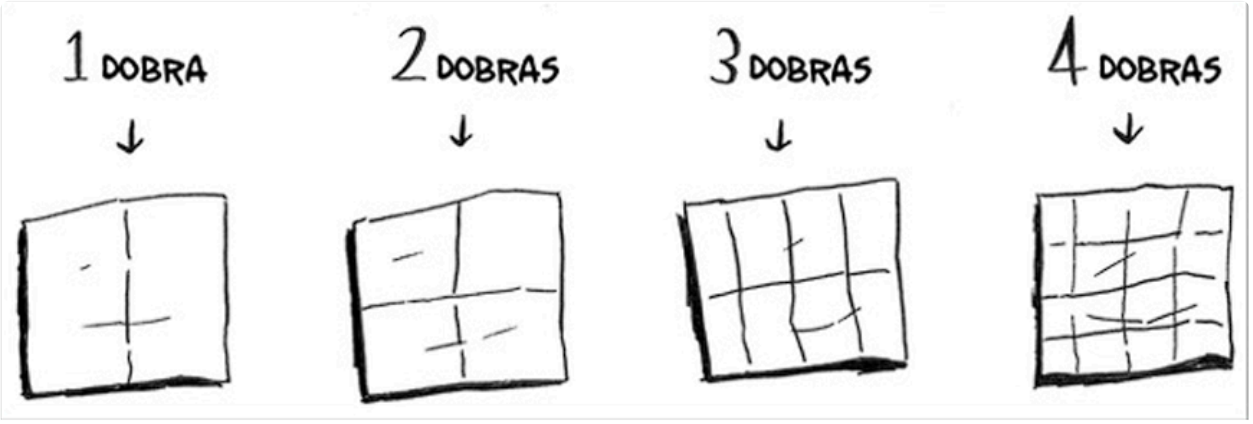
O formato da notação Big O.

Exemplo de uso da Big O: Suponha que você tenha que desenhar uma grade com 16 divisões. Com isso em mente, uma forma de desenhar essa grade de 16 divisões é desenhar uma divisão de cada vez. Nesse exemplo, desenhar uma divisão é uma operação, portanto, você precisa desenhar 16 divisões, ou seja, realizar 16 operações distintas.



Desenhando a grade executando uma divisão por vez.

Agora, digamos que eu pegue um papel vazio e o dobre quatro vezes consecutivas:



Ao fim das quatro dobras, igualmente, teremos 16 divisões no papel, no entanto com apenas 1/4 do número de operações necessárias do primeiro algoritmo. Assim, o algoritmo 1 tem tempo de execução O(n) e o algoritmo 2 tem tempo de execução O(log n), que é mais veloz.

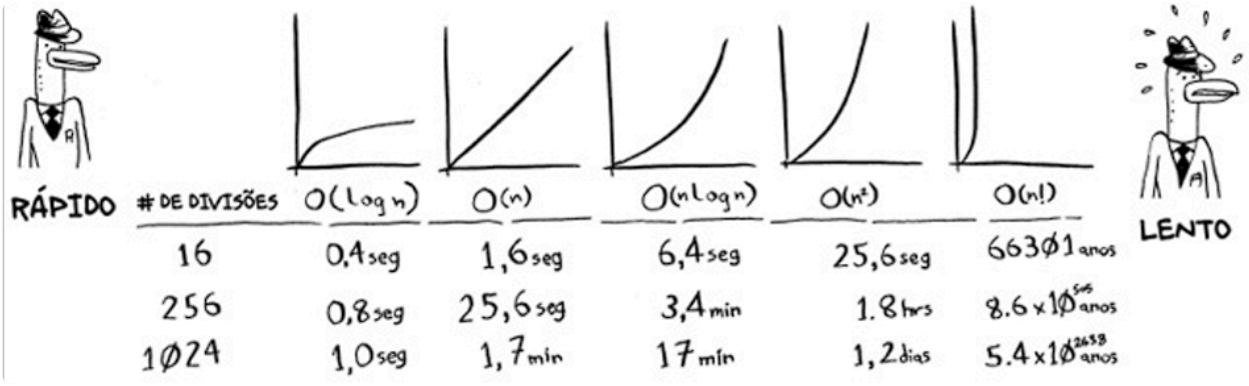
OBS: É importante mencionar que a **Big O leva em conta sempre a pior das hipóteses**, ou seja, o caso em que o algoritmo utiliza o maior número de operações possíveis para chegar em sua conclusão. Assim sendo, mesmo que uma pesquisa simples acerte logo na primeira tentativa a posição de um objeto de desejo (por esse ser o primeiro de uma lista ordenada), o tempo de execução desse algoritmo em Big O ainda seria O(n). Dessa forma, temos a garantia apenas que a pesquisa simples não terá um tempo de execução mais lento do que O(n), mas nada nos garante que ela possa cumprir seu papel em um tempo de execução menor.

Alguns exemplos comuns de tempo de execução Big O

Aqui temos cinco tempos de execução Big O que você encontrará bastante, ordenados do mais rápido para o mais lento.

- O(log n), também conhecido como *tempo logarítmico*. Exemplo: pesquisa binária.
- O(n), conhecido como *tempo linear*. Exemplo: pesquisa simples.
- O(n * log n). Exemplo: um algoritmo rápido de ordenação, como a ordenação quicksort (explicada no Capítulo 4).
- O(n^2). Exemplo: um algoritmo lento de ordenação, como a ordenação por seleção (explicada no Capítulo 2).
- O(n!). Exemplo: um algoritmo bastante lento, como o do caixeiro-viajante (explicado a seguir!).

Levando em consideração que podemos realizar, no máximo, dez operações em um segundo, aqui estariam gráficos ilustrando o desempenho dos cinco tempos de execução acima:

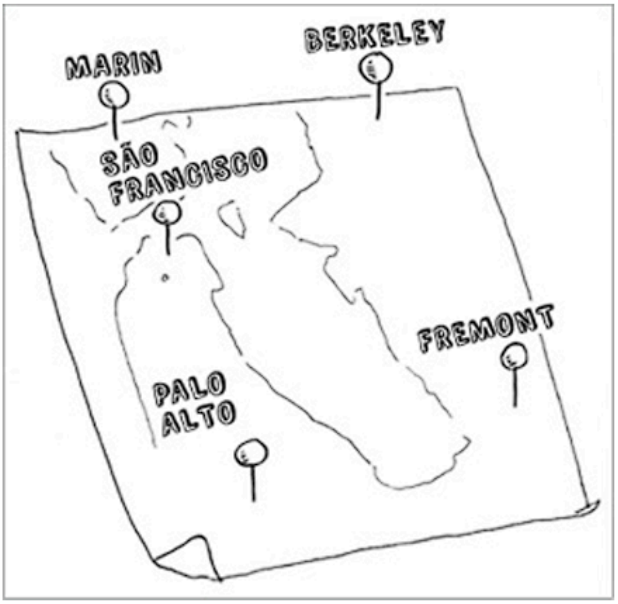


Lembrando que isso é uma simplificação, visto que, na realidade, você não pode converter um tempo de execução na notação Big O para um número de operações, mas a aproximação serve de exemplo para vislumbrarmos a realidade.

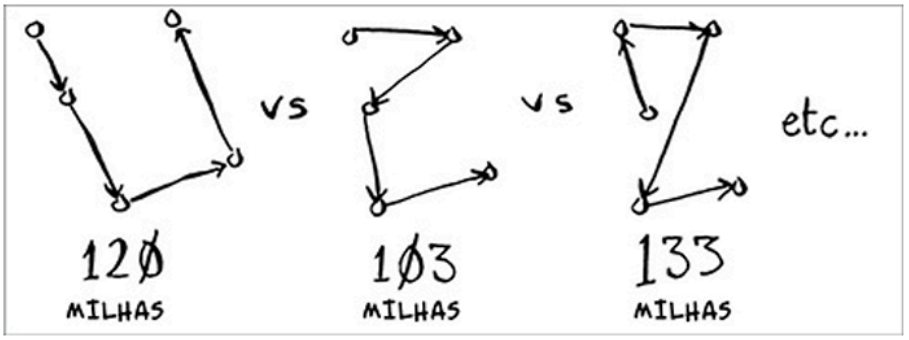
Pontos importantes:

- A rapidez de um algoritmo não é medida em segundos, mas pelo crescimento do número de operações.
- A notação Big O expressa o tempo de execução de um algoritmo.
- A notação Big O mede justamente a velocidade de um algoritmo no caso em que ele demanda o maior número possível de operações.
- O(log n) é mais rápido do que O(n), e O(log n) ca ainda mais rápido conforme a lista aumenta por ser um tempo de execução logarítmico.

Exemplo de algoritmo com tempo de execução fatorial O(n!): o caixeiro viajante.



O caixeiro, o qual chamarei de Opus, quer passar por todas as cidades percorrendo uma distância mínima. Podemos enxergar o problema da seguinte forma: analisar cada ordem possível de cidades para visitar.



Ele soma a distância total e escolhe o caminho de menor distância. Existem 120 permutações para cinco cidades, logo, precisa-se de 120 operações para resolver o problema de cinco cidades. Para seis cidades, precisa-se de 720 operações (ou 720 permutações). Para sete cidades são necessárias 5.050 operações!

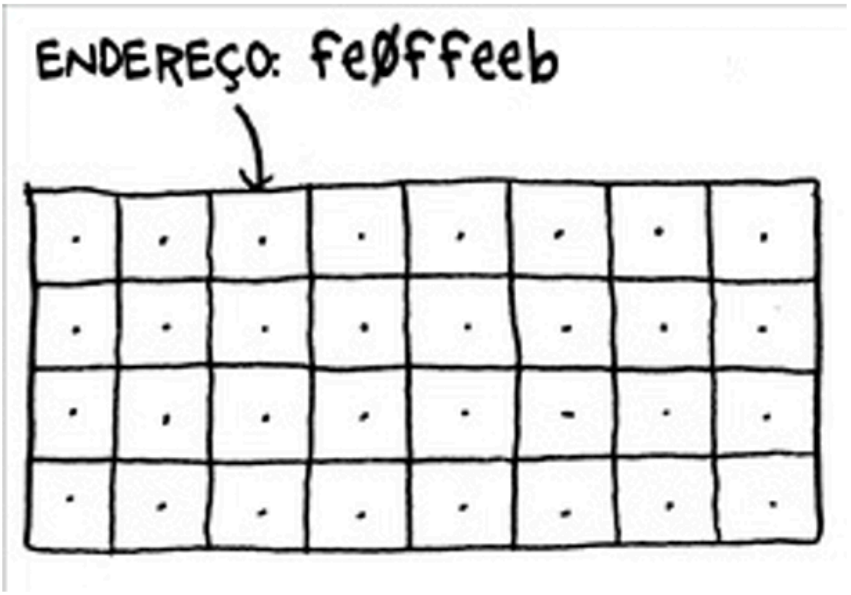
CIDADES	OPERAÇÕES
6	72Ø
7	5Ø4Ø
8	4Ø32Ø
...	...
15	1,3Ø7,674,368,ØØØ
...	...
3Ø	2,652,852,912,191,058,636,308,480,000,000

O número de operações aumenta drasticamente.

A quantidade de número de operações com 100 cidades é tanta que é impossível calcular a resposta em função do tempo, ou seja, o sol entrará em colapso antes que possamos achar a melhor via para o caixeiro viajar.

Ordenação por seleção

Para entendermos algoritmos de ordenação, é importante também termos conhecimento quanto a vetores e listas.



feØffeeb é o endereço de um slot na memória.

Para armazenarmos itens na memória do computador, geralmente pedimos a ele determinada quantidade de bytes para tal. Tomando como exemplo a linguagem C++, um inteiro equivale a 4 bytes e, portanto, quando vamos atribuir um valor a uma variável desse tipo, estamos separando 4 bytes na memória que irão armazenar esse inteiro.

Nesse sentido, quando precisamos armazenar múltiplos inteiros temos como possibilidades: 1 - armazená-los em várias variáveis separadas; 2 - utilizar uma estrutura de dados para facilitar esse processo. Com isso em mente, é óbvio que a primeira alternativa é extremamente custosa, além de dificultar o gerenciamento dos valores de interesse. Assim, é mais fácil utilizar vetores, listas, dicionários, ou tantas outras ferramentas para pedir ao computador por mais armazenamento de memória. No entanto, como o enfoque será em vetores e listas, irei explicar a diferença entre esses dois tipos de estrutura de dados.

Vetores

Um vetor é uma forma de armazenar contiguamente (de maneira adjacente/lado a lado) memória. Tomando como exemplo uma lista de afazeres, ao guardarmos nossas

tarefas em um vetor, teríamos um resultado semelhante a esse:

Agora, suponha que você queira adicionar mais uma tarefa. No entanto a próxima gaveta está ocupada por coisas de outra pessoa!

É como se você estivesse indo ao cinema com os seus amigos e encontrasse um lugar para sentar, mas outro amigo se juntasse a vocês e não houvesse lugar para ele. Assim sendo, vocês precisariam encontrar outra sequência de cadeiras que pudessem contemplar a todos vocês, possivelmente em outra fileira da sala de cinema. No entanto, e se outro amigo aparecesse? O processo teria que se repetir.
Uma forma de evitar essa fadiga seria reservando mais assentos (memória), no entanto, isso traria mais gasto a vocês, já que pode ser que nem todos possam comparecer ao filme, o que traria um desperdício de dinheiro e de espaço.

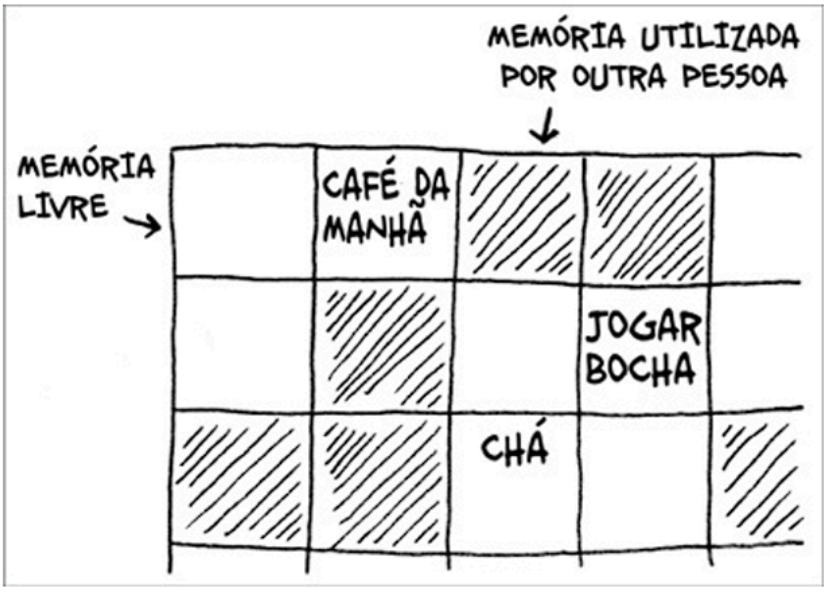
Dessa forma, ainda considerando o exemplo da lista de afazeres, esses seriam as possíveis desvantagens de se usar um vetor para seu gerenciamento:

- Você pode não precisar dos espaços extras que reservou; então a memória será desperdiçada. Você não está utilizando a memória, mas ninguém mais pode usá-la também.
- Você pode precisar adicionar mais de dez itens a sua lista de tarefas, então você terá de mover seus itens de qualquer maneira.

Listas encadeadas

As listas encadeadas, diferentemente dos vetores, não armazenam dados de maneira necessariamente contígua na memória, dessa forma, não há um tamanho pré-estabelecido de reserva de recursos computacionais (memória).

Com as listas encadeadas, seus itens podem estar em qualquer lugar da memória.



Cada item armazena o endereço do próximo item da lista. Um monte de endereços aleatórios de memória estão ligados.

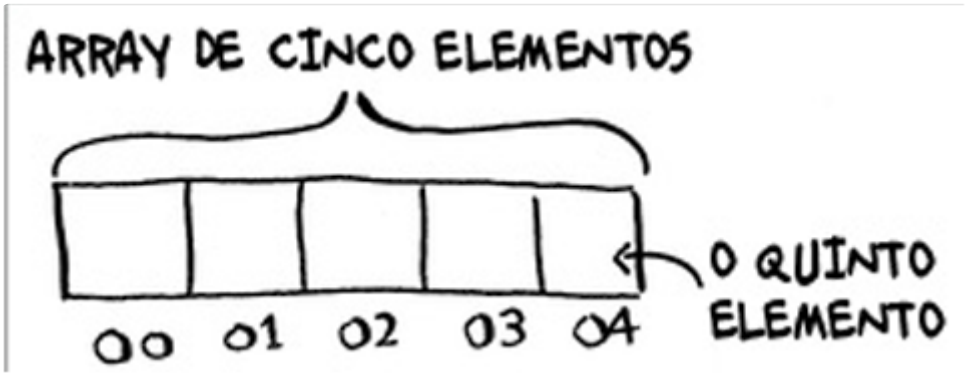
Endereços de memória ligados.

Cada um dos itens de interesse aponta para o próximo endereço na memória em que há um integrante da lista, até que não haja mais nenhum.

Seguindo o exemplo do cinema, ao usar uma lista encadeada você e seus amigos não se importam de estarem ou não juntos durante a sessão, o que seria como dizer “vamos nos dividir e assistir ao filme”. Desde que haja espaço no cinema/na memória, haverá espaço para a lista encadeada.

Essa flexibilidade na alocação de memória é uma vantagem das listas encadeadas, pois não exige que os itens sejam armazenados de forma contígua. No entanto, essa mesma característica também representa uma limitação: para acessar um item específico na lista, você precisa percorrer todos os itens anteriores, um por um, até chegar ao desejado. Isso ocorre porque cada item na lista só sabe onde está o próximo, sem informações diretas sobre os itens subsequentes.

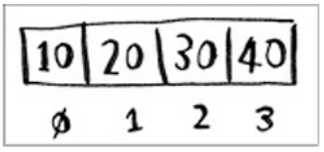
Listas encadeadas são ótimas se você quiser ler todos os itens, um de cada vez: você pode ler um item, seguir para o endereço do próximo item e fazer isso até o fim da lista. Mas se você quiser pular de um item para outro, as listas encadeadas são terríveis. Já com vetores, por estarem contíguos na memória, é bem mais fácil de acessar, por exemplo, o quinto item, já que não é necessário que passemos por cada um dos itens anteriores, visto que sabemos como apontar facilmente para seu endereço de memória: usando o índice 4 do vetor.



Vetores, diferentemente da lista encadeada, são ótimos se você deseja ler elementos aleatórios, pois pode encontrar qualquer elemento instantaneamente em um vetor bastando que saiba seu índice (que vai de zero até 1 - a capacidade máxima do vetor).

Levando em conta um vetor de capacidade máxima 4, temos seus índices indo de 0 até 3, como exemplificado abaixo:

Os elementos em um array são numerados. Essa numeração começa no 0, não no 1. Neste array, por exemplo, o número 20 está na posição 1.



	ARRAYS	LISTAS
LEITURA	O(1)	O(n)
INSERÇÃO	O(n)	O(1)
ELIMINAÇÃO	O(n)	O(1)

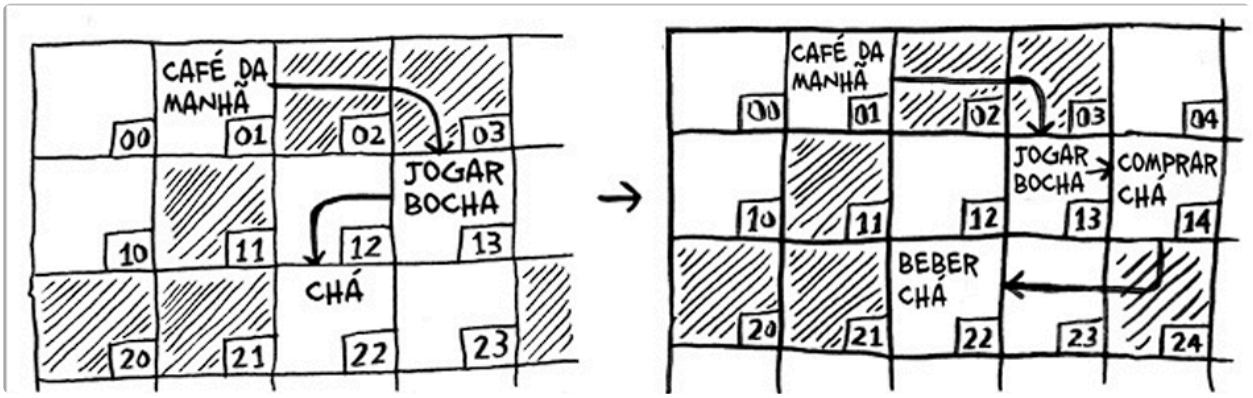
Assim, **vetores são mais vantajosos para operações de leitura** devido à facilidade de acesso às suas posições na memória e **listas (encadeadas) são melhores no contexto de inserção de dados**, por não demandarem uma alocação contígua de memória, sendo bem mais flexível nesse sentido.
Além disso, **deleção de elementos também é mais eficiente com o uso de listas**, visto que é necessário mudar apenas o endereço para o qual o elemento anterior da estrutura de dados está apontando. Já com vetores, tudo precisa ser movido quando um elemento é eliminado.

Inserção de elementos (Vetor vs. Lista encadeada)

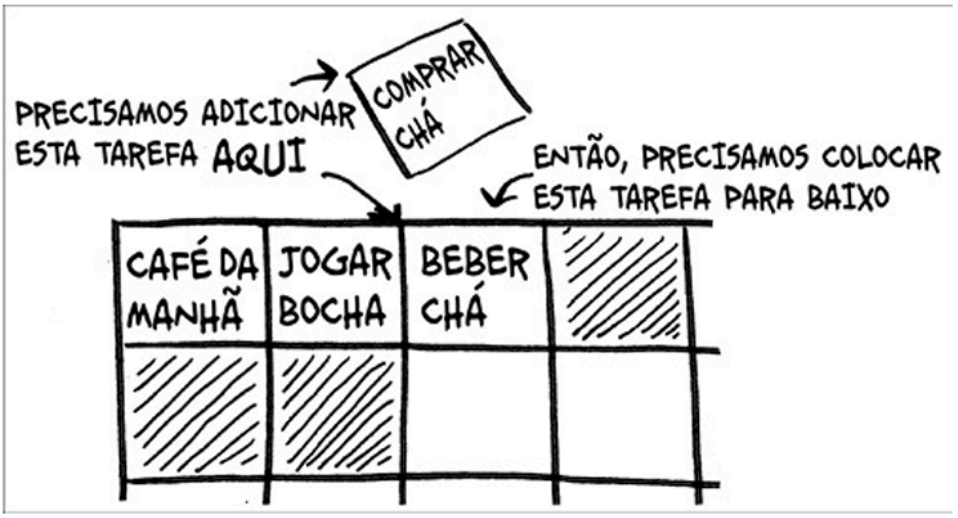
Para inserir elementos no meio de uma lista de tarefas, por exemplo, seria mais vantajoso o uso de um vetor ou de uma lista encadeada?

A resposta é simples: usando listas encadeadas, basta mudar o endereço para o qual o elemento anterior está apontando, já usando vetores, seria necessário encontrar outro endereço na memória que pudesse abarcar lado a lado todas as informações de afazeres (o que é bem mais complicado). Assim, listas encadeadas seriam melhores nesse contexto!

Exemplo com uma lista encadeada: Note que apenas alteramos para onde "jogar bocha" aponta, sendo fácil as operações de inserção/renomeio das demais tarefas.



Exemplo usando um vetor: Note que, como não há memória livre de maneira contígua que possa abarcar a nova tarefa, será necessário mover todos os dados para outro local na memória.



Se não houver espaço, pode ser necessário mover tudo para um novo local!

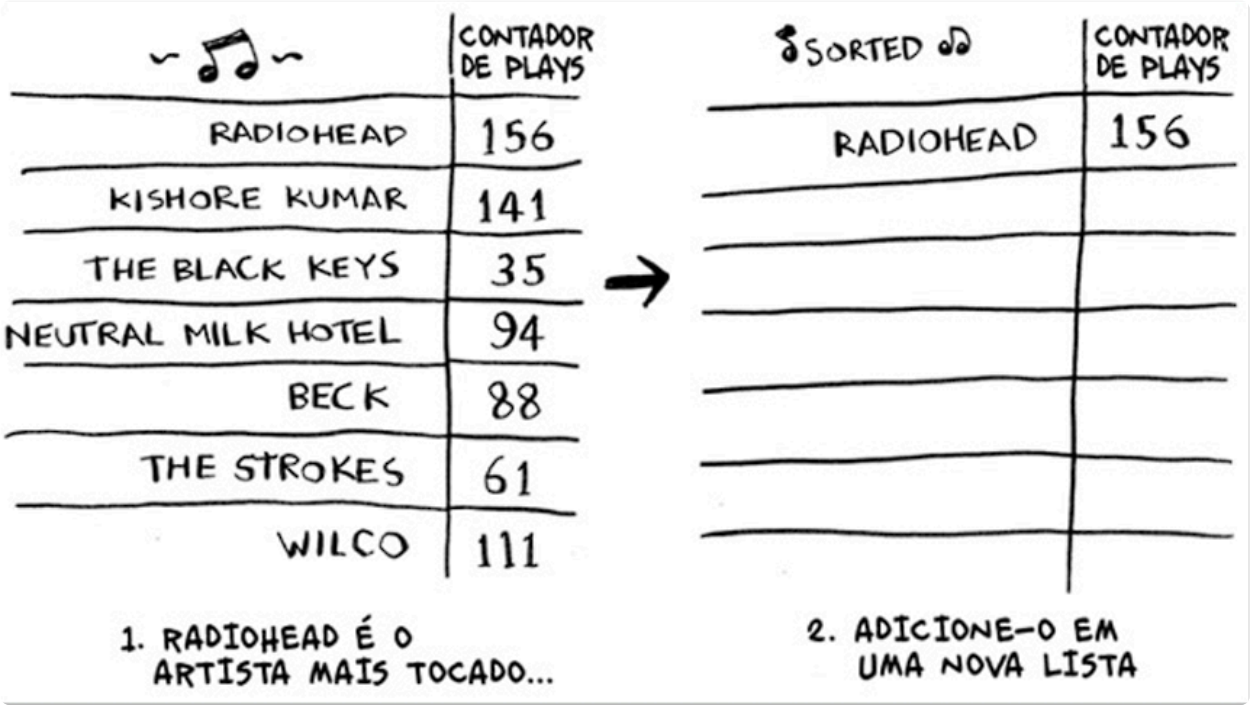
Qual é mais usado? Vetores ou listas encadeadas?

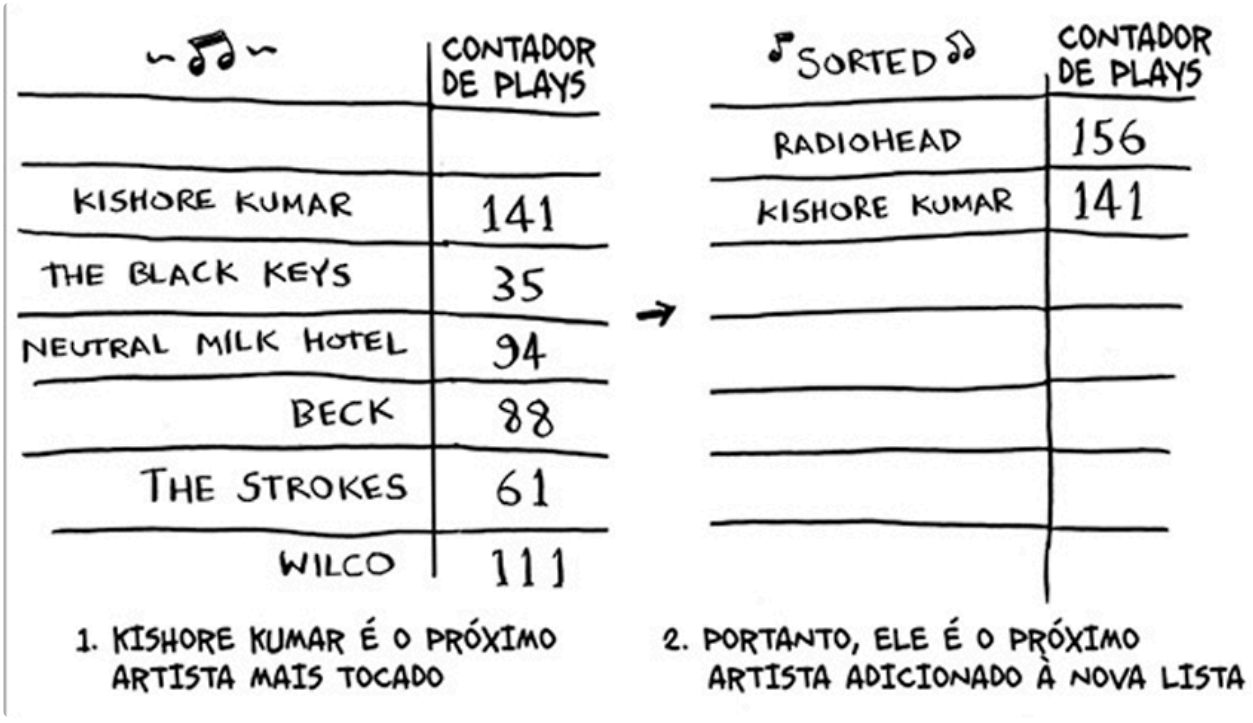
Os vetores são mais comuns porque permitem acesso aleatório. Existem dois tipos de acesso: o aleatório e o sequencial. O sequencial significa ler os elementos, um por um, começando pelo primeiro. Listas encadeadas só podem lidar com acesso sequencial.

Se você quiser ler o décimo elemento de uma lista encadeada, primeiro precisará ler os nove elementos anteriores para chegar ao endereço do décimo elemento. O aleatório permite que você pule direto para o décimo elemento. Muitos casos requerem o acesso aleatório, o que faz os arrays (vetores) serem mais utilizados. No entanto, é importante mencionar que ambos (vetores e listas) são utilizados para implementar outras estruturas de dados.

Retornando à ordenação por seleção...

Digamos que estamos querendo ordenar qual o artista mais tocado de uma lista de artistas, para isso, podemos utilizar o seguinte algoritmo: perpassar por cada elemento da lista (busca simples, tempo de execução O(n)) para encontrar o artista mais tocado, o retirar dessa lista de artistas e o adicionar a uma nova lista (que será nossa lista ordenada).





Esse processo iria se repetir diversas vezes, até termos então uma lista plenamente ordenada:

	CONTADOR DE PLAYS
RADIOHEAD	156
KISHORE KUMAR	141
WILCO	111
NEUTRAL MILK HOTEL	94
BECK	88
THE STROKES	61
THE BLACK KEYS	35

Como dito anteriormente, checar cada artista da lista original a fim de encontrar o com maior número de "plays" é uma tarefa com tempo de execução O(n), sendo n o número de itens (artistas) da lista. Dito isso, a cada nova iteração da busca um item é retirado e, portanto, a verificação passar de n elementos a n – 1, n - 2 … 2, até enfim um único elemento, com um resultado de O(n × ½ × n). No entanto, em Big O ignoramos constantes como ½, o que gera um resultado final de O(n²).

Isso tem tempo de execução $O(n \times n)$ ou $O(n^2)$.

Exemplo de algoritmo de ordenação por seleção em C++:

```
void selectionSort(Artista artistasVetor[], int tamanhoVetor){
    int posMenor;
    Artista aux;

    for(int i = 0; i<tamanhoVetor; i++){
        posMenor = i;

        for (int j=i+1; j < tamanhoVetor; j++) {
            if (artistasVetor[j].getContadorDePlays() >
artistasVetor[posMenor].getContadorDePlays()){
                posMenor = j;
            }
        }

        aux = artistasVetor[i];
        artistasVetor[i] = artistasVetor[posMenor];
        artistasVetor[posMenor] = aux;
    }
}
```

Pontos importantes:

- Quando se quer armazenar múltiplos elementos, usa-se um array ou uma lista.
- No array, todos os elementos são armazenados um ao lado do outro.
- Na lista, os elementos estão espalhados e um elemento armazena o endereço do próximo elemento.
- Arrays permitem leituras rápidas, mas tem inserções/deleções menos eficientes (devido à forma em que aloca a memória).
- Listas encadeadas permitem rápidas inserções e eliminações, mas tem acesso difícil, devido à sua natureza semelhante à lógica de ponteiros.

Recursão

É, resumidamente, a chamada de um método (função) dentro de outro método. A recursão é usada para tornar trechos de código mais simples de entender, não havendo nenhum benefício quanto ao desempenho ao ser utilizada. Em muitos casos, utilizar laços de repetição pode ser mais eficiente.

“Os loops podem melhorar o desempenho do seu programa. A recursão melhora o desempenho do seu programador. Escolha o que for mais importante para a sua situação.” - Leigh Caldwell, do Stack Overflow.

Algo importante de se notar em uma função recursiva é que precisamos sempre delimitar o momento de parada, para não acabarmos com execuções infinitas. Por isso, toda função recursiva tem duas partes: o caso-base e o caso recursivo.

- Caso recursivo: quando a função chama a si mesma.
- Caso-base: quando a função não chama a si mesma, interrompendo o loop.

Exemplo de recursão em C++:

```
#include <iostream>

using namespace std;

void contagemRecursiva(int contador){

    if(contador==0){ // Parada da recursão
        cout << "Atingimos o caso-base" << endl;
    }

    else {
        cout << "Estamos no caso recursivo" << endl;
        cout << contador << endl;
        contagemRecursiva(contador-1);
    }
}

int main(){

    contagemRecursiva(10);

    return 0;
}
```

Caso não houvesse o caso-base no método `contagemRecursiva()`, seria impresso infinitamente a variável `contador`, sendo, a cada impressão, subtraído 1 do seu valor total.

- Com caso-base:

```
gabrafo@gabrafo:~/Repositórios/Algoritmos&ED/implementacao_algoritmos$ g++ -Wall -Wextra recursao.cpp -o recursao
gabrafo@gabrafo:~/Repositórios/Algoritmos&ED/implementacao_algoritmos$ ./recursao
Estamos no caso recursivo
10
Estamos no caso recursivo
9
Estamos no caso recursivo
8
Estamos no caso recursivo
7
Estamos no caso recursivo
6
Estamos no caso recursivo
5
Estamos no caso recursivo
4
Estamos no caso recursivo
3
Estamos no caso recursivo
2
Estamos no caso recursivo
1
Atingimos o caso-base
```

- Sem caso-base:

```
gabrafo@gabrafo:~/Repositórios/Algoritmos&ED/implementacao_algoritmos$ g++ -Wall -Wextra recursao.cpp -o recursao
recursao.cpp: In function ‘void contagemRecursiva(int)’:
recursao.cpp:5:6: warning: infinite recursion detected [-Winfinite-recursion]
    5 | void contagemRecursiva(int contador){
      |      ^~~~~~~~~~~~~~~~~
recursao.cpp:9:22: note: recursive call
    9 |     contagemRecursiva(contador-1);
      |     ~~~~~~~~~~~~~~~~~^~~~~~~~~~~~
```

Primeiro, temos uma acusação do compilador que haverá uma recursão infinita.

```
Estamos no caso recursivo
-261853
Estamos no caso recursivo
-261854
Estamos no caso recursivo
-261855
Estamos no caso recursivo
-261856
Estamos no caso recursivo
-261857
Estamos no caso recursivo
-261858
Estamos no caso recursivo
-261859
Estamos no caso recursivo
-261860
Estamos no caso recursivo
-261861
Estamos no caso recursivo
-261862
Estamos no caso recursivo
-261863
Estamos no caso recursivo
-261864
Estamos no caso recursivo
-261865
Estamos no caso recursivo
Segmentation fault
```

Caso decidamos ignorar o aviso, o programa irá executar até chegar a uma falha de segmentação.